

Méthode d'évitement d'interblocage : Algorithme du banquier (Cas pratique)

L'algorithme du banquier est un algorithme d'évitement des interblocages qui s'applique dans le cas général où chaque type de ressources possède plusieurs instances. Le nom de l'algorithme a été choisi parce qu'il pourrait s'appliquer dans un système bancaire pour s'assurer que la banque ne prête jamais son argent disponible de telle sorte qu'elle ne puisse plus satisfaire tous ses clients.

Quand un nouveau processus entre dans le système, il doit déclarer le nombre maximal d'instances de chaque type de ressources dont il aura besoin. Ce nombre ne doit pas excéder le nombre total de ressources du système. Au cours de son exécution, quand un processus demande un ensemble de ressources, l'algorithme vérifie si cela gardera toujours le système dans un état sain.

Dans l'affirmative la demande est accordée, dans la négative la demande est retardée.

Soient **m** le nombre de types de ressources du système, et **n** le nombre de processus.

Pour fonctionner, l'algorithme maintient plusieurs structures de données :

- **Available** : C'est un Vecteur de longueur m indiquant le nombre de ressources disponibles de chaque type. Ainsi, si $\text{Available}[j]=k$, cela veut dire que le type de ressources R_j possède k instances disponibles.
- **Max** : C'est une matrice $n \times m$ définissant la demande maximale de chaque processus. Ainsi, Si $\text{Max}[i, j]=k$, cela veut dire que le processus P_i peut demander au plus k instances du type de ressources R_j .
- **Allocation** : C'est une matrice $n \times m$ définissant le nombre de ressources de chaque type de ressources actuellement alloué à chaque processus. Ainsi si $\text{Allocation}[i, j]=k$, cela veut dire que l'on a alloué au processus P_i k instances du type de ressources R_j .
- **Need** : C'est une matrice $n \times m$ indiquant les ressources restant à satisfaire à chaque processus. Ainsi, si $\text{Need}[i, j]=k$, cela veut dire que le processus P_i peut avoir besoin de k instances au plus du type de ressources R_j pour achever sa tâche.
- **Request** : C'est une matrice $n \times m$ indiquant les ressources supplémentaires que les processus viennent de demander. Ainsi, si $\text{Request}[i, j]=k$, cela veut dire que le processus P_i vient de demander k instances supplémentaires du type de ressources R_j .

De ce qui précède, on peut remarquer que :

1. Ces structures de données peuvent varier dans le temps, en taille et en valeur.

2. La matrice Need peut être calculée à partir des matrices Max et Allocation :

$$\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$$

Notations : Pour des raisons pratiques, on utilise les notations suivantes :

- **Allocation_i** : Vecteur désignant les ressources actuellement allouées au processus P_i.
- **Need_i** : Vecteur désignant les ressources supplémentaires que le processus P_i peut encore demander.
- **Request_i** : Vecteur désignant les ressources supplémentaires que le processus P_i vient de demander.
- $X \leq Y$: X et Y sont des vecteurs si $X[i] \leq Y[i]$ pour chaque i allant de 1 à n.

L'algorithme du Banquier peut être scindé en deux parties complémentaires : Un algorithme de requête de ressources et un algorithme de vérification si l'état du système est sain.

Nous donnons maintenant le code de chacune des deux parties.

Partie1 : Algorithme de requête de ressources

Début

/* Cet algorithme est appelé à chaque fois qu'un processus fait une demande de ressources

Etape 1 : Si Request_i ≤ Need_i

Alors Aller à l'étape 2

Sinon erreur : le processus a excédé ses besoins maximaux

Finsi

Etape 2 : Si Request_i ≤ Available

Alors Aller à l'étape 3

Sinon Attendre : les ressources ne sont pas disponibles.

Finsi

Etape 3 : Sauvegarder l'état du système (les matrices Available, Allocation et Need).

Allouer les ressources demandées par le processus P_i en modifiant l'état du système de la manière suivante :

Available := Available - Request_i

Allocation_i := Allocation_i + Request_i

Need_i := Need_i - Request_i

Si Verification_Etat_Sain = Vrai

Alors L'allocation est validée

Sinon L'allocation est annulée ; Restaurer l'ancien Etat du système

Finsi

Fin.

Partie2 : Le 2^{ème} algorithme est une fonction qui renvoie la valeur Vrai si le système est dans un état sain, Faux sinon.

Algorithme Verification_Etat_Sain

Début

Work : Tableau[m] de Entier ;

Finish : Tableau[n] de Logique ;

Etape 1 : Work :=Available

Finish :=Faux ;

Etape 2 : Trouver i tel que : Finish[i]=faux et Needi<=Work

Si un tel i n'existe pas aller à l'étape 4.

Etape 3 : Work :=Work + Allocationi

Finish[i] :=Vrai

Aller à l'étape 2

Etape 4 : Si Finish=Vrai (pour tout i)

Alors Verification_Etat_Sain :=Vrai

Sinon Verification_Etat_Sain :=Faux

Finsi

Fin.

Exemple explicatif : Un système possède 5 processus (P0, P1, P2, P3, P4) et 3 types de ressources (A, B, C). Le type de ressources A possède 10 instances, le type de ressources B possède 5 instances et le type de ressources C possède 7 instances. A l'instant T0, l'état des ressources du système est décrit par les matrices Allocation, Max et Available suivantes :

❖ **Matrice : Allocation**

	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

❖ **Matrice : Max**

	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

❖ Vecteur : Available

A	B	C
3	3	2

Le contenu de la matrice Need peut être déduit par calcul, $Need = Max - Allocation$

❖ Matrice : Need

	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

On peut vérifier que le système est dans un état sain. En effet, la séquence <P1, P3, P4, P2, P0> est saine.

Supposons qu'à l'instant T1 le processus P1 demande une instance supplémentaire du type de ressources A et deux instances du type de ressources C.

Nous avons alors : $Request1 = (1, 0, 2)$.

Afin de décider si la requête peut être immédiatement accordée, on doit d'abord vérifier que **$Request1 \leq Available$** , ce qui est vrai. On enregistre l'état du système (le contenu des matrices), puis on supposera que la requête a été satisfaite et on arrive à l'état suivant :

Matrice : Allocation

	A	B	C
P0	0	1	0
P1	3	0	2
P2	3	0	2
P3	2	1	1
P4	0	0	2

Matrice : Need

	A	B	C
P0	7	4	3
P1	0	2	0
P2	6	0	0
P3	0	1	1
P4	4	3	1

Matrice : Available

A	B	C
2	3	0

On doit alors déterminer si le nouvel état est sain en appliquant l'algorithme de vérification de l'état sain, ce qui est vrai (la séquence $\langle P1, P3, P4, P0, P2 \rangle$ est saine). On peut donc accorder la demande de ressource faite par P1.

A l'instant T2, une requête (3, 3, 0) arrive du processus P4. Cette requête est immédiatement rejetée parce que les ressources ne sont pas disponibles.

A l'instant T3, une requête (0, 2, 0) arrive du processus P0. Cette requête ne sera pas accordée parce que l'état obtenu est malsain.